

md to html

```
import markdown
import requests

def convert(nom_md: str,nom_html: str):
    with open(nom_md, 'r') as f:
        text = f.read()
    html = markdown.markdown(text)
    with open(nom_html, 'w') as f:
        f.write(html)
```

Partie 1

```
import markdown
import requests
from md_to_html import convert

def get_node(id: int) -> dict:
    url = "https://www.openstreetmap.org/api/0.6/node/" + str(id) + ".json"
    response = requests.get(url)
    json_data = response.json()
    return json_data

def node_to_md(data: dict, filename: str) -> None:
    donnees = data.get("elements")[0]
    conteneur_md = "# " + str(donnees.get('tags').get('name')) + "\n\n"
    attributs = donnees.get("tags")
    if attributs:
        conteneur_md += "## Informations\n"
        for clef, valeur in attributs.items():
            conteneur_md += "* " + str(clef) + ": " + str(valeur) + "\n"
    with open(filename, 'w', encoding="utf-8") as f:
        f.write(conteneur_md)

def fiche_osm(id: int) -> None :
    donnees = get_node(id)
    data= donnees.get("elements")[0]
    nom = str(data.get('tags').get('name'))
    node_to_md(donnees, nom + ".md")
    convert(nom + ".md", nom + ".html")
```

Partie 2

```

import requests
from md_to_html import convert

def get_dataset(query: str) -> dict:
    """Récupère les données de l'api overpass pour une query donnée et retourne
    un dictionnaire."""
    url = "https://overpass-api.de/api/interpreter?data=" + str(query)
    response = requests.get(url)
    response.raise_for_status()
    data = response.json()
    return data

query = """
[out:json];
area["name"="Bordeaux"]->.a;
(
node["amenity"="restaurant"](area.a);
node["amenity"="fast_food"](area.a);
);
out tags;
"""

def compute_statistics(dataset: dict) -> dict:
    """Calcule les statistiques sur les informations (le pourcentage de
    fastfood/restaurant, le pourcentage de si les horaires sont renseignés,
    le pourcentage d'accessibilité fauteuil, les types de cuisines les plus
    fréquents)
    des restaurant/fastfoods et retourne un dict avec les stats."""
    elements = dataset["elements"]
    total = len(elements)

    stats = {
        "total": total,
        "fast_food": 0,
        "restaurant": 0,
        "with_hours": 0,
        "with_wheelchair": 0,
        "cuisines": {},
        "no_cuisine": 0 }

    for informations in elements:
        tags = informations.get("tags", {})

        if tags.get("amenity") == "fast_food":
            stats["fast_food"] += 1
        if tags.get("amenity") == "restaurant":
            stats["restaurant"] += 1

        if "opening_hours" in tags:
            stats["with_hours"] += 1

```

```

    if "wheelchair" in tags:
        stats["with_wheelchair"] += 1

    if "cuisine" in tags:
        type_cuisine = tags["cuisine"]
        if type_cuisine in stats["cuisines"]:
            stats["cuisines"][type_cuisine] += 1
        else:
            stats["cuisines"][type_cuisine] = 1
    else:
        stats["no_cuisine"] += 1
return stats

def dataset_to_md(dataset: dict, filename: str) -> None:
    """ Cette fonction prend le jeu de données au format JSON téléchargé depuis
    l'API Overpass
    et produit un fichier Markdown avec toutes les statistiques principales. """
    stats = compute_statistics(dataset)
    total = stats["total"]

    with open(filename, "w", encoding="utf-8") as f:
        f.write("# Statistiques des restaurants et fast-foods à Bordeaux\n\n")

        f.write(f"- Nombre total d'établissements : {total}\n")
        f.write(f"- Fast-foods : {round(stats['fast_food'] / total * 100, 1)}
%\n")
        f.write(f"- Restaurants classiques : {round(stats['restaurant'] / total
* 100, 1)} %\n")
        f.write(f"- Établissements avec horaires renseignés :
{round(stats['with_hours'] / total * 100, 1)} %\n")
        f.write(f"- Accessibilité fauteuil renseignée :
{round(stats['with_wheelchair'] / total * 100, 1)} %\n\n")

        f.write("## Les 10 types de cuisines les plus présentes\n")

        cuisines_list = []
        for nom, nb in stats["cuisines"].items():
            cuisines_list.append((nom, nb))

        for i in range(len(cuisines_list)):
            for o in range(i + 1, len(cuisines_list)):
                if cuisines_list[o][1] > cuisines_list[i][1]:
                    cuisines_list[i], cuisines_list[o] = cuisines_list[o],
cuisines_list[i]

        top10_cuisines = cuisines_list[:10]

        for nom, nb in top10_cuisines:
            percent = round(nb / total * 100, 1)
            f.write(f"- {nom} : {percent} %\n")

        pourcentage_nc = round(stats["no_cuisine"] / total * 100, 1)
        f.write("\n## Cuisine non renseignée\n")

```

```
f.write(f"- Non renseignée : {pourcentage_nc} %\n")

def infos_locales(query: str) -> None:
    """Cette fonction utilise toutes les fonctions précédentes pour télécharger
    un jeu de données
    et construire un fichier Markdown et un fichier HTML présentant ces données
    et leur statistiques."""

    print("Téléchargement des données...")
    donnée = get_dataset(query)
    dataset_to_md(donnée, "stats_bordeaux.md")
    convert("stats_bordeaux.md", "stats_bordeaux.html")
    print("Terminé ! Fichiers créés : stats_bordeaux.md et
    stats_bordeaux.html")

infos_locales(query)
```